

1. The four stages are, Preprocessing, Compilation, Assembly, and Linking.
Preprocessing expands include and define directives and strips comments producing pure C source with no macros or directives left in it. Compilation translates that preprocessed source into assembly code for the target architecture on the course Pi image. Assembly turns that assembly into machine code, producing an object file (.o) compiled binary code, but not yet a runnable program. And last, linking, combines one or more object files with any needed libraries into a single executable, resolving reference such as calls to printf across file boundaries.
2. -Wall adds additional warnings, while -Wextra adds further warnings beyond -Wall. Finally Werror makes warnings become errors.
3. The -c gcc compilation will let the prompt know whether you are stopping to create an object file or linking the program.
4. For a makefile rule the target is the file that the rule proceeds. The prerequisites are the files that the target depends on, the recipe is the shell command(s) that the makefile runs to build from the prereqs.
5. Rebuilding an up to date target is pointless as the separate compilations deem it unnecessary, It avoids this by using the dependency graph which it uses recursively and can go up the logic.
6. Commits only change local git repository, the repo copy doesn't exist anywhere until it is pushed. Which allows submissions to be graded as they can be seen by the teacher. Commit allows us to preserve our history and push lets the instructor see and grade.